

Integración continua con GitHub Actions

5. Integración Continua con GitHub Actions

¿Qué problema resuelve la integración continua?

Un equipo de cuatro personas hace merge en main sin saber si el código de un compañero rompe los tests de otra. Descubrirlo días después —o en producción— cuesta mucho más que detectarlo en el acto.

Integración continua (CI)

Ejecutar comprobaciones automáticas sobre el código cada vez que alguien propone un cambio: compilar, pasar tests, revisar el estilo. Si algo falla, avisa antes de llegar a main.

git push / PR



GitHub Actions



¿Pasan
los tests?

Sí 

Merge permitido

No 

Merge bloqueado, aviso al autor

GitHub Actions: qué es y qué lo dispara

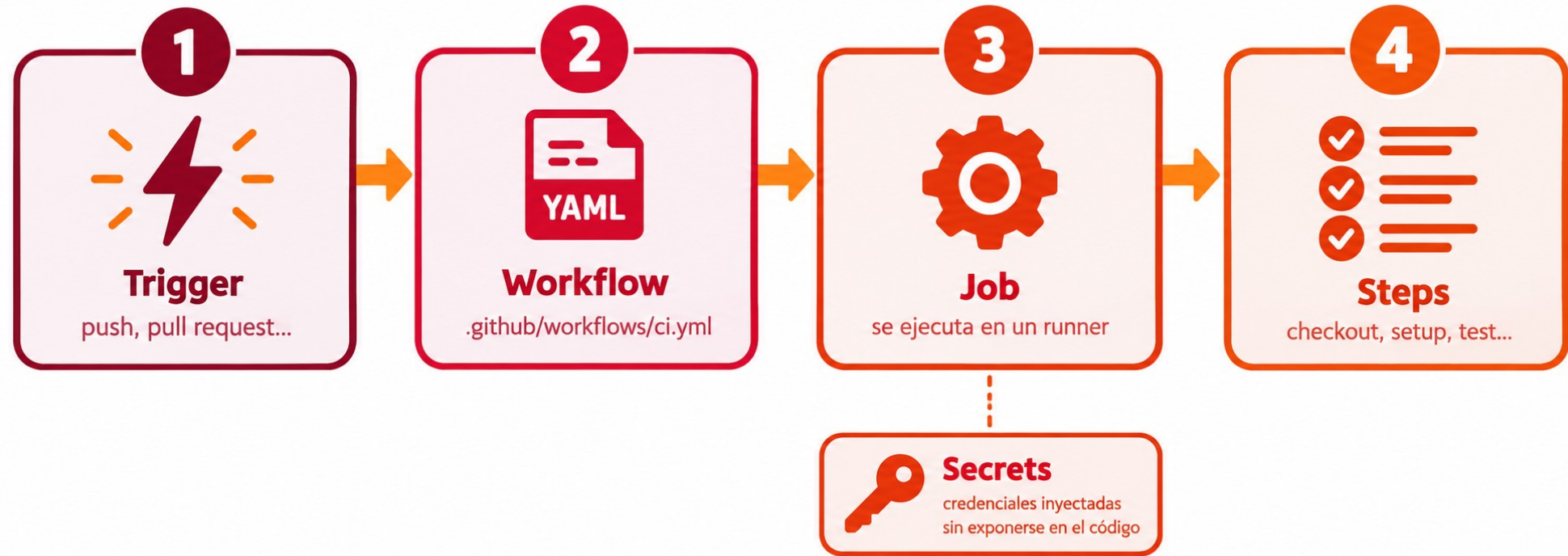
El sistema de automatización integrado en GitHub

Define workflows que se ejecutan en respuesta a eventos del repositorio, en la nube de GitHub, sin instalar nada en tu máquina.

Eventos más habituales que disparan un workflow

Evento	Cuándo ocurre
push	Alguien sube commits a una rama
pull_request	Se abre, actualiza o cierra una PR
schedule	Un temporizador (como un cron job)
workflow_dispatch	El usuario lo activa manualmente desde la web
release	Se publica una nueva versión del proyecto

Conceptos clave: las piezas que encajan



Workflow

El YAML en
`.github/workflows/`

Job

Bloque de pasos en una
máquina virtual

Runner

La VM donde corre: Ubuntu,
Windows, macOS

Action

Tarea reutilizable del
Marketplace

Estructura de un workflow: ficheros YAML

Viven en `.github/workflows/` — puedes tener varios ficheros

```
mi-proyecto-ci/  
├── .github/  
│   └── workflows/  
│       └── ci.yml          ← se ejecuta en cada push a main  
├── src/main/java/.../Calculadora.java  
├── src/test/java/.../CalculadoraTest.java  
└── pom.xml
```

```
name: CI - Build y Tests  
on:  
  push:  
    branches: [main]  
jobs:  
  build:  
    runs-on: ubuntu-latest  
    steps:  
      - name: Descargar el código fuente  
        uses: actions/checkout@v7
```

Evita referencias como `@main` en una Action: apuntan al desarrollo más reciente y pueden cambiar sin que hayas tocado tu workflow. Usa la versión mayor estable, como `@v7`.

Cada sección del YAML tiene su función

Sección	Qué hace
name	El nombre que verás en la interfaz de GitHub
on	Qué eventos activan el workflow — puedes listar varios
jobs	Uno o más jobs; por defecto se ejecutan en paralelo
runs-on	El tipo de runner — ubuntu-latest es el más común
steps	Pasos en orden; si uno falla, los siguientes no se ejecutan (salvo if: always())

Node.js 24 es el runtime por defecto de las Actions JavaScript desde junio de 2026: conviene mantener actualizadas las Actions oficiales.

El proyecto de ejemplo: Calculadora

Calculadora.java

```
public class Calculadora {  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
    public double dividir(double a, double b) {  
        if (b == 0) throw new  
            IllegalArgumentException("...");  
        return a / b;  
    }  
}
```

CalculadoraTest.java

```
class CalculadoraTest {  
    private final Calculadora calc =  
        new Calculadora();  
    @Test  
    void sumarDosPositivos() {  
        assertEquals(5, calc.sumar(2, 3));  
    }  
    @Test  
    void dividirPorCeroLanzaExcepcion() {  
        assertThrows(  
            IllegalArgumentException.class,  
            () -> calc.dividir(10, 0));  
    }  
}
```

Cuatro operaciones y cinco tests que las verifican. Cuando se hace push con ci.yml en su sitio, GitHub Actions ejecuta automáticamente todos los pasos del workflow.

Qué hace cada step con la Calculadora

1

1. `actions/checkout@v7`

Clona el repositorio en el runner, igual que git clone. Sin él, los pasos siguientes no encontrarían ni el pom.xml ni las clases Java.

2

2. `actions/setup-java@v6`

Instala el JDK 21 de Temurin. cache: 'maven' guarda ~/.m2 entre ejecuciones.

3

3. `mvn compile -B`

Compila las clases. Un error de sintaxis detiene el workflow aquí, sin llegar a los tests.

4

4. `mvn test -B`

Ejecuta los cinco tests. Si alguno falla, Maven termina con error y el workflow se marca como fallido.

5

5. `upload-artifact, if: always()`

Guarda el informe de Surefire como artifact descargable — se ejecuta aunque los tests fallen.

El panel Actions de GitHub

● **Naranja**

El workflow está en ejecución ahora.

✓ **Verde**

Todos los steps han pasado.

✗ **Rojo**

Algún step ha fallado.

Vista general → un run → un job → sus steps con el log completo

```
[INFO] Running com.ejemplo.CalculadoraTest  
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0  
[INFO] BUILD SUCCESS
```

Si un step falla, su fondo se vuelve rojo. Busca la última línea antes de «Process completed with exit code 1»: ahí está el problema.

Cuando el workflow falla: tres orígenes distintos

Error en el código

El más frecuente. sumarDosPositivos falla: expected <5> but was <-1>. Solución: corrige, commit y push.

Error en el workflow

El YAML tiene un problema o una Action no existe: Unable to resolve action. Solución: corrige el .yml.

Error de entorno

Al runner le falta algo: JAVA_HOME is not set. Solución: añade el step de instalación que falta.

Pasa en local, falla en el runner

La causa más frecuente es una diferencia de entorno: versión de Java, dependencia no instalada, o un fichero local que no está en el repositorio. El runner parte de cero cada vez.

El Marketplace de Actions

No tienes que escribir todo desde cero. El Marketplace contiene miles de Actions reutilizables de la comunidad y de empresas.

Action	Para qué sirve
<code>actions/checkout@v7</code>	Clona el repositorio en el runner. Siempre el primer step.
<code>actions/setup-java@v6</code>	Instala un JDK concreto (Temurin, Zulu, Corretto...).
<code>actions/setup-node@v7</code>	Instala Node.js y npm.
<code>actions/cache@v6</code>	Guarda en caché dependencias (Maven, npm...).
<code>actions/upload-artifact@v7</code>	Guarda ficheros del runner para descargarlos después.

Verifica las Actions de terceros

Las oficiales (actions/...) son de confianza. Las de terceros ejecutan código en tu runner con acceso a tus secrets: comprueba que el repositorio es conocido.

Secretos: credenciales sin exponer

Nunca escribas contraseñas, tokens o claves API en el YAML: ese fichero vive en el repositorio y cualquiera que lo clone lo verá. Los Secrets son variables cifradas que el workflow puede usar sin que su valor aparezca en ningún log.

Crear un secret

Settings → **Secrets**

and variables → Actions

Usarlo en el workflow

```
env:  
  MAVEN_PASSWORD:  
    ${ secrets.MAVEN_PASSWORD }
```

Los secrets no llegan a PRs de forks externos

Por seguridad, evita que alguien cree un fork y extraiga tus credenciales con echo. En PRs de tu propio repositorio, sí funcionan.

Ideas clave

1

CI: ejecutar comprobaciones automáticas en cada cambio, antes de que lleguen a main.

2

GitHub Actions se configura con ficheros YAML en `.github/workflows/`.

3

Un workflow se activa con un event, contiene jobs que corren en un runner, y cada job tiene steps.

4

Los steps usan `uses:` para invocar una Action del Marketplace, o `run:` para un comando de shell.

5

Si falla: puede ser el código, el workflow (YAML) o el entorno del runner — cada uno se diagnostica distinto.

6

Los secrets guardan credenciales cifradas: nunca escribas contraseñas directamente en el YAML.

7

`if: always()` en un step garantiza que se ejecute aunque los anteriores fallen — útil para guardar informes.